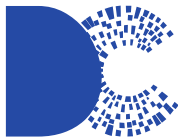




UNIVERSIDADE
FEDERAL DE
SERGIPE



DEPARTAMENTO
DE COMPUTAÇÃO

Ferramentas de desenvolvimento

Interface Hardware/Software

Bruno Prado

Departamento de Computação / UFS

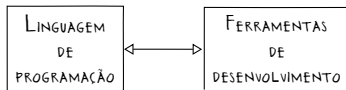
Introdução

- ▶ As ferramentas de desenvolvimento criam a infraestrutura básica para compilação e depuração

LINGUAGEM
DE
PROGRAMAÇÃO

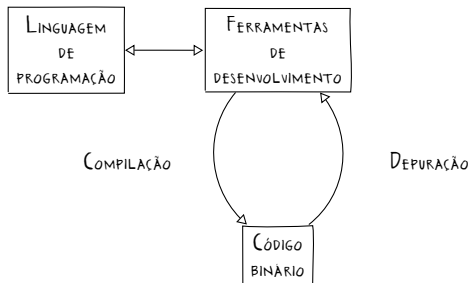
Introdução

- ▶ As ferramentas de desenvolvimento criam a infraestrutura básica para compilação e depuração



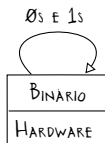
Introdução

- ▶ As ferramentas de desenvolvimento criam a infraestrutura básica para compilação e depuração



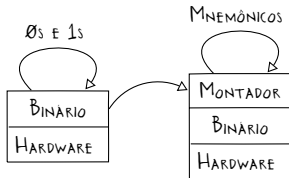
Introdução

► Processo de *bootstrapping*



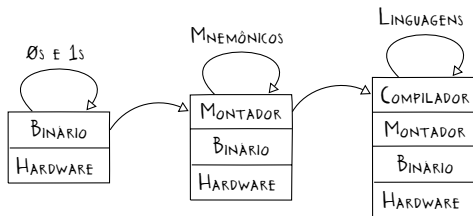
Introdução

► Processo de *bootstrapping*



Introdução

► Processo de *bootstrapping*



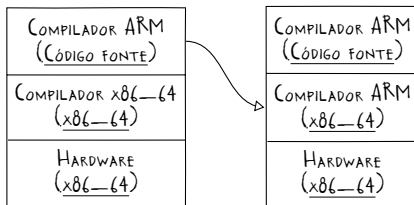
Introdução

► Compilação cruzada

COMPILADOR ARM (<u>CÓDIGO FONTE</u>)
COMPILADOR x86-64 (<u>x86-64</u>)
HARDWARE (<u>x86-64</u>)

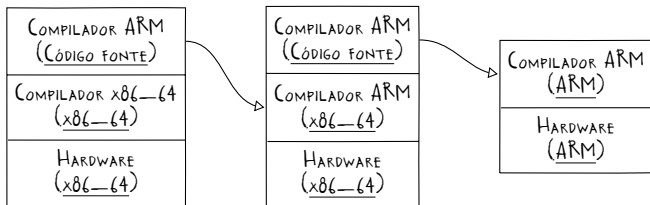
Introdução

► Compilação cruzada



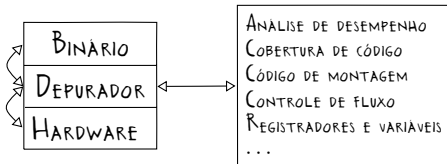
Introdução

► Compilação cruzada



Introdução

► Depuração



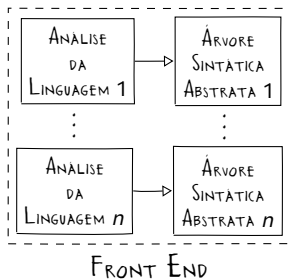
Compilação

- ▶ GCC ou LLVM?
 - ▶ Código aberto
 - ▶ GPLv3 × NCSA Open Source License

	GCC	LLVM
Arquitetura	–	+
Compatibilidade	+	–
Documentação	–	+
Linguagens	+	–
Otimização de código	+	+

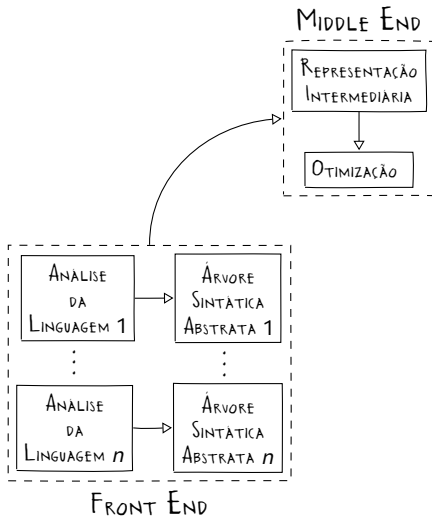
Compilação

► Arquitetura dos compiladores



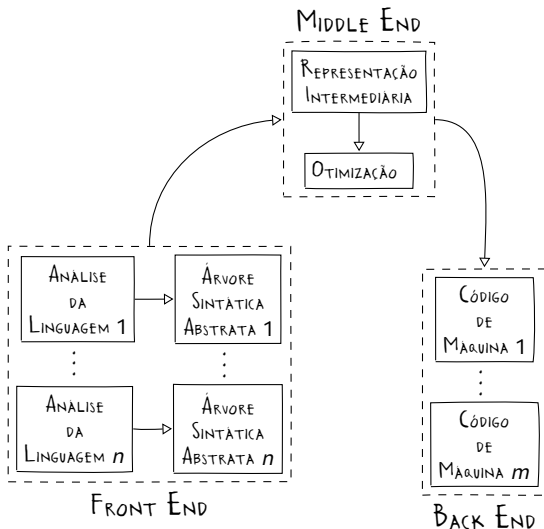
Compilação

► Arquitetura dos compiladores



Compilação

► Arquitetura dos compiladores



Compilação

► Implementação em C

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c = %u\n", c);
7     return 0;
8 }
```

Compilação

► Árvore Sintática Abstrata (LLVM)

```
$ clang -S -Xclang -ast-dump exemplo.c
```

Compilação

► Árvore Sintática Abstrata (LLVM)

```
...
|
|-FunctionDecl 0x5617a8c01640 <exemplo.c:3:1, line:8:1> line:3:5 main
'int ()'
|-CompoundStmt 0x5617a8c0ce38 <col:12, line:8:1>
| -DeclStmt 0x5617a8c0cbb0 <line:4:2, col:30>
| | -VarDecl 0x5617a8c0c990 <col:2, col:15> col:11 used a 'uint32_t':
unsigned int' cinit
| | | -ImplicitCastExpr 0x5617a8c0ca18 <col:15> 'uint32_t': 'unsigned i
nt' <IntegralCast>
| | | | -IntegerLiteral 0x5617a8c0c9f8 <col:15> 'int' 1
| | | -VarDecl 0x5617a8c0ca40 <col:2, col:22> col:18 used b 'uint32_t':
unsigned int' cinit
| | | | -ImplicitCastExpr 0x5617a8c0cac8 <col:22> 'uint32_t': 'unsigned i
nt' <IntegralCast>
| | | | | -IntegerLiteral 0x5617a8c0caa8 <col:22> 'int' 2
| | | -VarDecl 0x5617a8c0caf0 <col:2, col:29> col:25 used c 'uint32_t':
unsigned int' cinit
| | | | -ImplicitCastExpr 0x5617a8c0cb78 <col:29> 'uint32_t': 'unsigned i
nt' <IntegralCast>
| | | | | -IntegerLiteral 0x5617a8c0cb58 <col:29> 'int' 0
| | -BinaryOperator 0x5617a8c0cc78 <line:5:2, col:10> 'uint32_t': 'unsig
...

```

Compilação

► Árvore Sintática Abstrata (LLVM)

```
...
d int' lvalue Var 0x5617a8c0c990 'a' 'uint32_t':'unsigned int'
|   '-ImplicitCastExpr 0x5617a8c0cc40 <col:10> 'uint32_t':'unsigned i
nt' <LValueToRValue>
|   '-DeclRefExpr 0x5617a8c0cc08 <col:10> 'uint32_t':'unsigne
d int' lvalue Var 0x5617a8c0ca40 'b' 'uint32_t':'unsigned int'
|   '-CallExpr 0x5617a8c0cd90 <line:6:2, col:22> 'int'
|   '-ImplicitCastExpr 0x5617a8c0cd78 <col:2> 'int (*)'(const char *, ..
.)' <FunctionToPointerDecay>
|   '-DeclRefExpr 0x5617a8c0cc98 <col:2> 'int (const char *, ...)' Fu
nction 0x5617a8be37b0 'printf' 'int (const char *, ...)'
|   '-ImplicitCastExpr 0x5617a8c0cdd8 <col:9> 'const char *' <NoOp>
|   '-ImplicitCastExpr 0x5617a8c0cdc0 <col:9> 'char *' <ArrayToPointe
rDecay>
|   '-StringLiteral 0x5617a8c0ccf8 <col:9> 'char [8]' lvalue "c = %
u\n"
|   '-ImplicitCastExpr 0x5617a8c0cdf0 <col:21> 'uint32_t':'unsigned int'
' <LValueToRValue>
|   '-DeclRefExpr 0x5617a8c0cd18 <col:21> 'uint32_t':'unsigned int' l
value Var 0x5617a8c0caf0 'c' 'uint32_t':'unsigned int'
'-ReturnStmt 0x5617a8c0ce28 <line:7:2, col:9>
'-IntegerLiteral 0x5617a8c0ce08 <col:9> 'int' 0
$
```

Compilação

► Implementação em C

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c = %u\n", c);
7     return 0;
8 }
```

Compilação

► Representação intermediária (LLVM)

```
$ clang -S -emit-llvm exemplo.c
```

Compilação

► Representação intermediária (LLVM)

```
$ clang -S -emit-llvm exemplo.c  
$ cat exemplo.ll
```

Compilação

► Representação intermediária (LLVM)

```
$ clang -S -emit-llvm exemplo.c
$ cat exemplo.ll
...
define dso_local i32 @main() #0 {
    %1 = alloca i32, align 4
    %2 = alloca i32, align 4
    %3 = alloca i32, align 4
    %4 = alloca i32, align 4
    store i32 0, i32* %1, align 4
    store i32 1, i32* %2, align 4
    store i32 2, i32* %3, align 4
    store i32 0, i32* %4, align 4
    %5 = load i32, i32* %2, align 4
    %6 = load i32, i32* %3, align 4
    %7 = add i32 %5, %6
    store i32 %7, i32* %4, align 4
    %8 = load i32, i32* %4, align 4
    %9 = call i32 (@i8*, ...) @printf(i8* @getelementptr inbounds ([8 x i8]
), [8 x i8]* @.str, i64 0, i64 0), i32 %8)
    ret i32 0
}
...
$
```

Compilação

► Implementação em C

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c = %u\n", c);
7     return 0;
8 }
```

Compilação

► Código de máquina (GCC)

```
$ gcc -Wall exemplo.c -o exemplo.elf
```

Compilação

► Código de máquina (GCC)

```
$ gcc -Wall exemplo.c -o exemplo.elf  
$ objdump --disassemble -M intel exemplo.elf
```

Compilação

► Código de máquina (GCC)

```
...
0000000000001139 <main>:
    1139:    55                push    rbp
    113a:    48 89 e5          mov     rbp, rsp
    113d:    48 83 ec 10       sub     rsp, 0x10
    1141:    c7 45 f4 01 00 00 00 mov     DWORD PTR [rbp-0xc], 0x1
    1148:    c7 45 f8 02 00 00 00 mov     DWORD PTR [rbp-0x8], 0x2
    114f:    c7 45 fc 00 00 00 00 mov     DWORD PTR [rbp-0x4], 0x0
    1156:    8b 55 f4          mov     edx, DWORD PTR [rbp-0xc]
    1159:    8b 45 f8          mov     eax, DWORD PTR [rbp-0x8]
    115c:    01 d0            add     eax, edx
    115e:    89 45 fc          mov     DWORD PTR [rbp-0x4], eax
    1161:    8b 45 fc          mov     eax, DWORD PTR [rbp-0x4]
    1164:    89 c6            mov     esi, eax
    1166:    48 8d 3d 97 0e 00 00 lea     rdi, [rip+0xe97]
# 2004 <_IO_stdin_used+0x4>
    116d:    b8 00 00 00 00    mov     eax, 0x0
    1172:    e8 b9 fe ff ff    call    1030 <printf@plt>
    1177:    b8 00 00 00 00    mov     eax, 0x0
    117c:    c9              leave
    117d:    c3              ret
    117e:    66 90           xchg    ax, ax
...
```

Exercício

- ▶ Compilar e instalar as ferramentas de desenvolvimento GCC e LLVM
 - ▶ Reproduza os passos vistos na aula
 - ▶ Faça uma análise comparativa destas ferramentas
 - ▶ Demonstre os resultados com exemplos

Depuração

► Compilação com suporte para depuração

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c = %u\n", c);
7     return 0;
8 }
```

```
$ clang -Wall -g exemplo.c -o exemplo.elf
```

Depuração

► Compilação com suporte para depuração

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c = %u\n", c);
7     return 0;
8 }
```

```
$ gcc -Wall -g exemplo.c -o exemplo.elf
```

Depuração

► Controle de fluxo (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c = %u\n", c);
7     return 0;
8 }
```

```
$ gdb exemplo.elf
GNU gdb (GDB) 9.2
...
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from exemplo.elf...
(gdb)
```

Depuração

► Controle de fluxo (LLDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c=%u\n", c);
7     return 0;
8 }
```

```
$ lldb exemplo.elf
(lldb) target create "exemplo.elf"
Current executable set to 'exemplo.elf' (x86_64).
(lldb)
```

Depuração

► Controle de fluxo (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c=%u\n", c);
7     return 0;
8 }
```

```
(gdb) break main
Breakpoint 1 at 0x1141: file exemplo.c, line 4.
```

Depuração

► Controle de fluxo (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c=%u\n", c);
7     return 0;
8 }
```

```
(gdb) break main
Breakpoint 1 at 0x1141: file exemplo.c, line 4.
(gdb) run
Starting program: exemplo.elf

Breakpoint 1, main () at exemplo.c:4
4         uint32_t a = 1, b = 2, c = 0;
```

Depuração

► Controle de fluxo (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c=%u\n", c);
7     return 0;
8 }
```

```
(gdb) break main
Breakpoint 1 at 0x1141: file exemplo.c, line 4.
(gdb) run
Starting program: exemplo.elf

Breakpoint 1, main () at exemplo.c:4
4         uint32_t a = 1, b = 2, c = 0;
(gdb) step
5         c = a + b;
```

Depuração

► Controle de fluxo (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c=%u\n", c);
7     return 0;
8 }
```

```
(gdb) break main
Breakpoint 1 at 0x1141: file exemplo.c, line 4.
(gdb) run
Starting program: exemplo.elf

Breakpoint 1, main () at exemplo.c:4
4         uint32_t a = 1, b = 2, c = 0;
(gdb) step
5         c = a + b;
(gdb) backtrace
#0  main () at exemplo.c:5
(gdb)
```

Depuração

► Inspeção de código (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c = %u\n", c);
7     return 0;
8 }
```

(gdb) disassemble

Dump of assembler code for function main:

```
...
=> 0x000055555555156 <+29>:    mov     -0xc(%rbp),%edx
0x000055555555159 <+32>:    mov     -0x8(%rbp),%eax
0x00005555555515c <+35>:    add     %edx,%eax
...
0x000055555555172 <+57>:    callq   0x55555555030 <printf@plt>
0x000055555555177 <+62>:    mov     $0x0,%eax
0x00005555555517c <+67>:    leaveq
0x00005555555517d <+68>:    retq
End of assembler dump.
```

Depuração

► Inspeção de variáveis (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c=%u\n", c);
7     return 0;
8 }
```

```
(gdb) print a
$1 = 1
(gdb) print b
$2 = 2
(gdb) print c
$3 = 0
```

Depuração

► Inspeção de variáveis (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c = %u\n", c);
7     return 0;
8 }
```

```
(gdb) print a
$1 = 1
(gdb) print b
$2 = 2
(gdb) print c
$3 = 0
(gdb) step
6         printf("c = %u\n", c);
```

Depuração

► Inspeção de variáveis (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c = %u\n", c);
7     return 0;
8 }
```

```
(gdb) print a
$1 = 1
(gdb) print b
$2 = 2
(gdb) print c
$3 = 0
(gdb) step
6         printf("c = %u\n", c);
(gdb) print c
$4 = 3
(gdb)
```

Depuração

► Inspeção de registradores (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c = %u\n", c);
7     return 0;
8 }
```

(gdb) disassemble

...

=> 0x00005555555555161 <+40>: mov -0x4(%rbp),%eax

0x00005555555555164 <+43>: mov %eax,%esi

...

Depuração

► Inspeção de registradores (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c = %u\n", c);
7     return 0;
8 }
```

```
(gdb) disassemble
...
=> 0x0000555555555161 <+40>:    mov     -0x4(%rbp),%eax
    0x0000555555555164 <+43>:    mov     %eax,%esi
...
(gdb) print $pc
$5 = (void (*)(void)) 0x555555555161 <main+40>
```

Depuração

► Inspeção de registradores (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c = %u\n", c);
7     return 0;
8 }
```

```
(gdb) disassemble
...
=> 0x00005555555555161 <+40>:    mov     -0x4(%rbp),%eax
    0x00005555555555164 <+43>:    mov     %eax,%esi
...
(gdb) print $pc
$5 = (void (*)(void)) 0x5555555555161 <main+40>
(gdb) stepi
0x00005555555555164      6          printf("c = %u\n", c);
```

Depuração

► Inspeção de registradores (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c = %u\n", c);
7     return 0;
8 }
```

```
...
=> 0x000055555555161 <+40>:    mov     -0x4(%rbp),%eax
    0x000055555555164 <+43>:    mov     %eax,%esi
...
(gdb) print $pc
$5 = (void (*)(void)) 0x55555555161 <main+40>
(gdb) stepi
0x000055555555164      6          printf("c = %u\n", c);
(gdb) continue
Continuing.
c = 3
[Inferior 1 (process 15764) exited normally]
```

Depuração

► Inspeção de registradores (GDB)

```
1 #include <stdio.h>
2 #include <stdint.h>
3 int main() {
4     uint32_t a = 1, b = 2, c = 0;
5     c = a + b;
6     printf("c=%u\n", c);
7     return 0;
8 }
```

```
0x00005555555555164 <+43>:    mov    %eax,%esi
...
(gdb) print $pc
$5 = (void (*)(void)) 0x5555555555161 <main+40>
(gdb) stepi
0x00005555555555164      6          printf("c = %u\n", c);
(gdb) continue
Continuing.
c = 3
[Inferior 1 (process 15764) exited normally]
(gdb) quit
$
```

Depuração

► Avaliação de desempenho (gprof)

```
1  #include <stdio.h>
2  #include <stdint.h>
3  uint64_t fatorial_i(uint32_t n) {
4      uint64_t i, r = 1;
5      for(i = 2; i <= n; i++) r = r * i;
6      return r;
7  }
8  uint64_t fatorial_r(uint32_t n) {
9      if(n <= 1) return 1;
10     else return n * fatorial_r(n - 1);
11 }
12 int main() {
13     ...
14     ...
15     ...
16     ...
17     ...
18     ...
19     ...
20 }
```

Depuração

► Avaliação de desempenho (gprof)

```
1  #include <stdio.h>
2  #include <stdint.h>
...
12 int main() {
13     uint64_t fi, fr, i;
14     for(i = 0; i < 1000000; i++) {
15         fi = fatorial_i(15);
16         fr = fatorial_r(15);
17     }
18     printf("fi = %lu, fr = %lu\n", fi, fr);
19     return 0;
20 }
```

```
$ gcc -Wall -g -pg exemplo.c -o exemplo.elf
```

Depuração

► Avaliação de desempenho (gprof)

```
1  #include <stdio.h>
2  #include <stdint.h>
...
12 int main() {
13     uint64_t fi, fr, i;
14     for(i = 0; i < 1000000; i++) {
15         fi = fatorial_i(15);
16         fr = fatorial_r(15);
17     }
18     printf("fi = %lu, fr = %lu\n", fi, fr);
19     return 0;
20 }
```

```
$ gcc -Wall -g -pg exemplo.c -o exemplo.elf
$ ./exemplo.elf
fi = 1307674368000, fr = 1307674368000
$
```

Depuração

► Avaliação de desempenho (gprof)

```
1  #include <stdio.h>
2  #include <stdint.h>
...
12 int main() {
13     uint64_t fi, fr, i;
14     for(i = 0; i < 1000000; i++) {
15         fi = fatorial_i(15);
16         fr = fatorial_r(15);
17     }
18     printf("fi = %lu, fr = %lu\n", fi, fr);
19     return 0;
20 }
```

```
$ gprof exemplo.elf
```

```
...
% cumulative self      self      total
time  seconds seconds  calls ns/call ns/call  name
60.97   0.06   0.06 1000000  60.97  60.97 fatorial_r
40.65   0.10   0.04 1000000  40.65  40.65 fatorial_i
...
```

Depuração

► Avaliação de desempenho (perf)

```
1  #include <stdio.h>
2  #include <stdint.h>
...
12 int main() {
13     uint64_t fi, fr, i;
14     for(i = 0; i < 1000000; i++) {
15         fi = fatorial_i(15);
16         fr = fatorial_r(15);
17     }
18     printf("fi = %lu, fr = %lu\n", fi, fr);
19     return 0;
20 }
```

```
$ gcc -Wall -g exemplo.c -o exemplo.elf
```

Depuração

► Avaliação de desempenho (perf)

```
1  #include <stdio.h>
2  #include <stdint.h>
...
12 int main() {
13     uint64_t fi, fr, i;
14     for(i = 0; i < 1000000; i++) {
15         fi = fatorial_i(15);
16         fr = fatorial_r(15);
17     }
18     printf("fi = %lu, fr = %lu\n", fi, fr);
19     return 0;
20 }
```

```
$ gcc -Wall -g exemplo.c -o exemplo.elf
$ perf record ./exemplo.elf
fi = 1307674368000, fr = 1307674368000
[ perf record: Woken up 1 times to write data ]
[ perf record: Captured and wrote 0.012 MB perf.data (295 samples) ]
$
```

Depuração

► Avaliação de desempenho (perf)

```
1  #include <stdio.h>
2  #include <stdint.h>
...
12 int main() {
13     uint64_t fi, fr, i;
14     for(i = 0; i < 1000000; i++) {
15         fi = fatorial_i(15);
16         fr = fatorial_r(15);
17     }
18     printf("fi = %lu, fr = %lu\n", fi, fr);
19     return 0;
20 }
```

```
$ perf report
Samples: 295 of event 'cycles:u', Event count (approx.): 257527146
Overhead Command Shared Object Symbol
60.34% exemplo.elf exemplo.elf [.] fatorial_r
35.41% exemplo.elf exemplo.elf [.] fatorial_i
4.13% exemplo.elf exemplo.elf [.] main
...
```

Depuração

► Cobertura de código (gcov)

```
1  #include <stdio.h>
2  #include <stdint.h>
3  int main() {
4      uint32_t a = 0;
5      if(a + 1 == 3) {
6          a = a * 7;
7          a = a << 11;
8      }
9      else {
10         a = 1;
11     }
12     printf("a_=%u\n", a);
13     return 0;
14 }
```

Depuração

► Cobertura de código (gcov)

```
1  #include <stdio.h>
2  #include <stdint.h>
3  int main() {
4      uint32_t a = 0;
5      if(a + 1 == 3) {
6          a = a * 7;
7          a = a << 11;
8      }
9      else {
10         a = 1;
11     }
12     printf("a_=%u\n", a);
13     return 0;
14 }
```

Este trecho do código nunca será executado!

Depuração

► Cobertura de código (gcov)

```
$ gcc -Wall --coverage -g exemplo.c -o exemplo.elf
```

Depuração

► Cobertura de código (gcov)

```
$ gcc -Wall --coverage -g exemplo.c -o exemplo.elf  
$ ./exemplo.elf  
a = 1
```

Depuração

► Cobertura de código (gcov)

```
$ gcc -Wall --coverage -g exemplo.c -o exemplo.elf
$ ./exemplo.elf
a = 1
$ gcov exemplo.elf-exemplo.c
File 'exemplo.c'
Lines executed:75.00% of 8
Creating 'exemplo.c.gcov'
```

Depuração

► Cobertura de código (gcov)

```
Lines executed:75.00% of 8
Creating 'exemplo.c.gcov'

$ cat exemplo.c.gcov
-: 0:Source:exemplo.c
-: 0:Graph:exemplo.gcno
-: 0:Data:exemplo.gcda
-: 0:Runs:1
-: 1:#include <stdio.h>
-: 2:#include <stdint.h>
1: 3:int main() {
1: 4:     uint32_t a = 0;
1: 5:     if(a + 1 == 3) {
####: 6:         a = a * 7;
####: 7:         a = a << 11;
-: 8:     }
-: 9:     else {
1: 10:         a = 1;
-: 11:     }
1: 12:     printf("a = %u\n", a);
1: 13:     return 0;
-: 14:}
```

Depuração

► Cobertura de código (gcov)

```
Lines executed:75.00% of 8
Creating 'exemplo.c.gcov'

$ cat exemplo.c.gcov
-: 0:Source:exemplo.c
-: 0:Graph:exemplo.gcno
-: 0:Data:exemplo.gcda
-: 0:Runs:1
-: 1:#include <stdio.h>
-: 2:#include <stdint.h>
1: 3:int main() {
1: 4:     uint32_t a = 0;
1: 5:     if(a + 1 == 3) {
####: 6:         a = a * 7;
####: 7:         a = a << 11;
-: 8:     }
-: 9:     else {
1: 10:         a = 1;
-: 11:     }
1: 12:     printf("a = %u\n", a);
1: 13:     return 0;
-: 14:}

$ exit
```

Exercício

- ▶ Reproduzir os experimentos vistos nesta aula
 - ▶ Obter manual de comandos do GDB/LLDB
 - ▶ Comparar resultados entre *gprof* e *perf*
 - ▶ Realizar cobertura de código em implementações mais complexas para entender esta funcionalidade ajuda no processo de depuração do código fonte